Neural Networks

Multilayer perceptrons and backpropagation

Outline

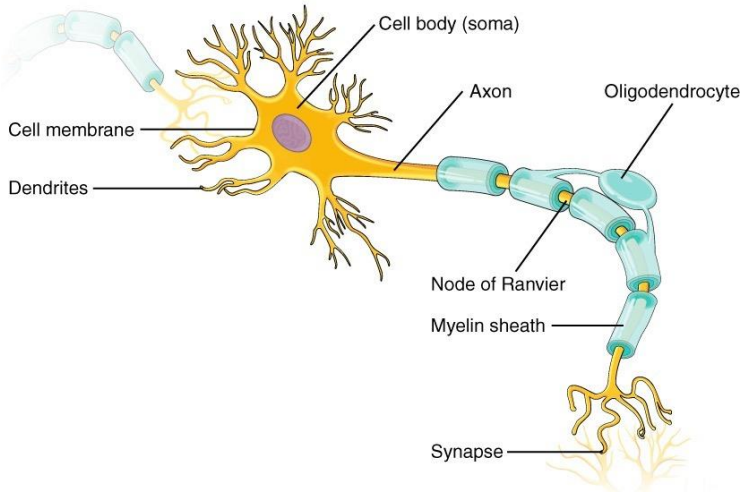
- Perceptrons and multilayer perceptrons (MLPs)
- Activation functions and forward inference
- Loss functions, gradient descent and backpropagation
- Worked examples and practice

Later topics in the module

Convolutional and recurrent networks, autoencoders, generative adversarial networks and transformers

Neural Networks

- Biological neurons receive signals through connected cells.
- Synapses change with experience.
- Artificial networks learn by adjusting connection weights.



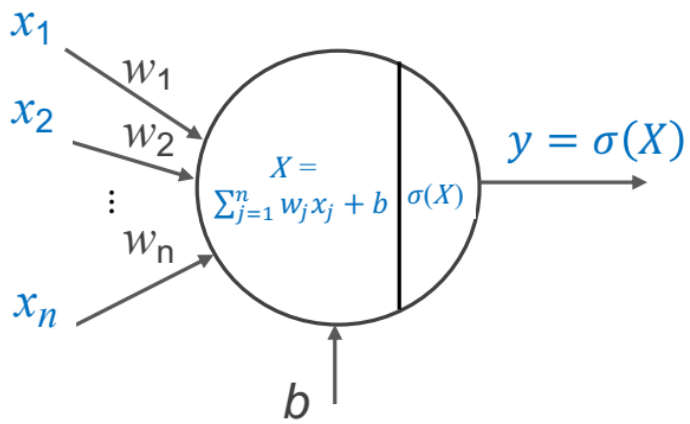
Neuron illustration: OpenStax / Wikimedia Commons / CC BY 4.0

Perceptrons

A perceptron combines weighted inputs and a bias.

$$z = \sum_i w_i x_i + b$$

$$y = \text{step}(z) = 1 \quad \text{if } z > 0, \quad \text{otherwise } 0$$

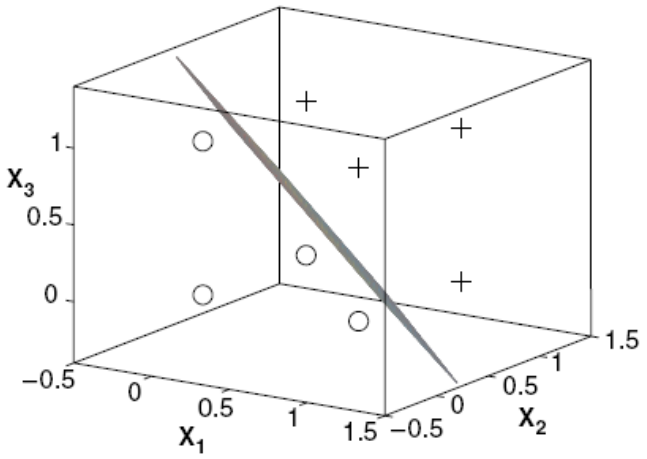


The bias b shifts the threshold. The threshold on $\sum w_i x_i$ is $-b$.

Perceptrons as Linear Classifiers

$$z = w^T x + b$$

- Boundary: $w^T x + b = 0$.
- A line in 2D, a plane in 3D, a hyperplane in higher dimensions.

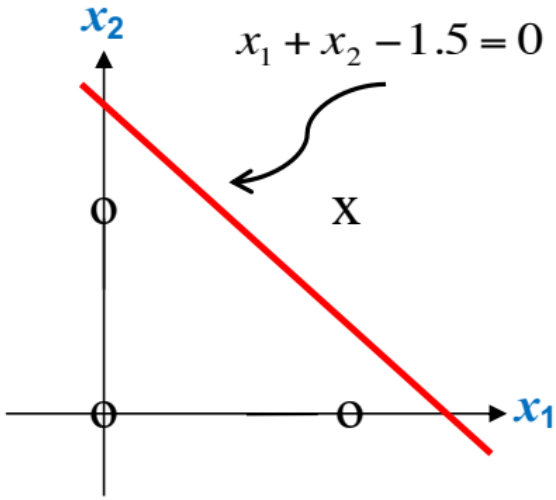


Bias as an extra weight: $x_0 = 1$ and $w_0 = b$.

Perceptron Example: AND

$$y = \text{step}(x_1 + x_2 - 1.5)$$

x_1	x_2	z	y
0	0	−1.5	0
0	1	−0.5	0
1	0	−0.5	0
1	1	0.5	1

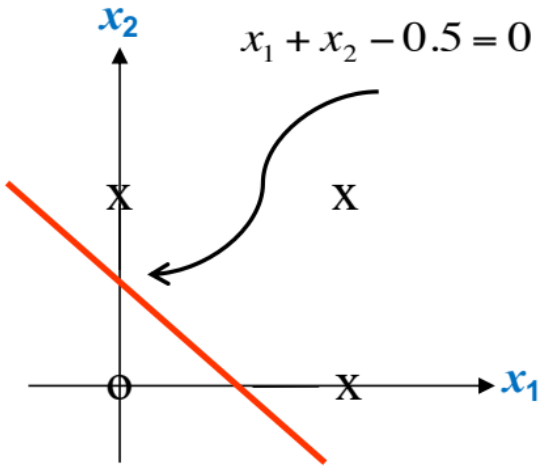


Only (1, 1) lies on the positive side of the boundary.

Perceptron Example: OR

$$y = \text{step}(x_1 + x_2 - 0.5)$$

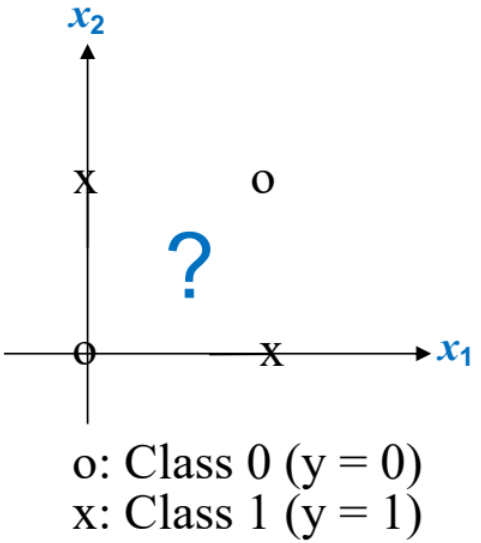
x_1	x_2	z	y
0	0	−0.5	0
0	1	0.5	1
1	0	0.5	1
1	1	1.5	1



Changing only the bias changes the classification rule.

Why One Perceptron Cannot Model XOR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



The positive cases require $w_1 > -b$ and $w_2 > -b$.

With $b \leq 0$, their sum makes $w_1 + w_2 + b > 0$.

But (1, 1) must be class 0. No single boundary works.

Forward Example 1: XOR with an MLP

$h_1 = \text{step}(x_1 + x_2 - 0.5)$ [OR]

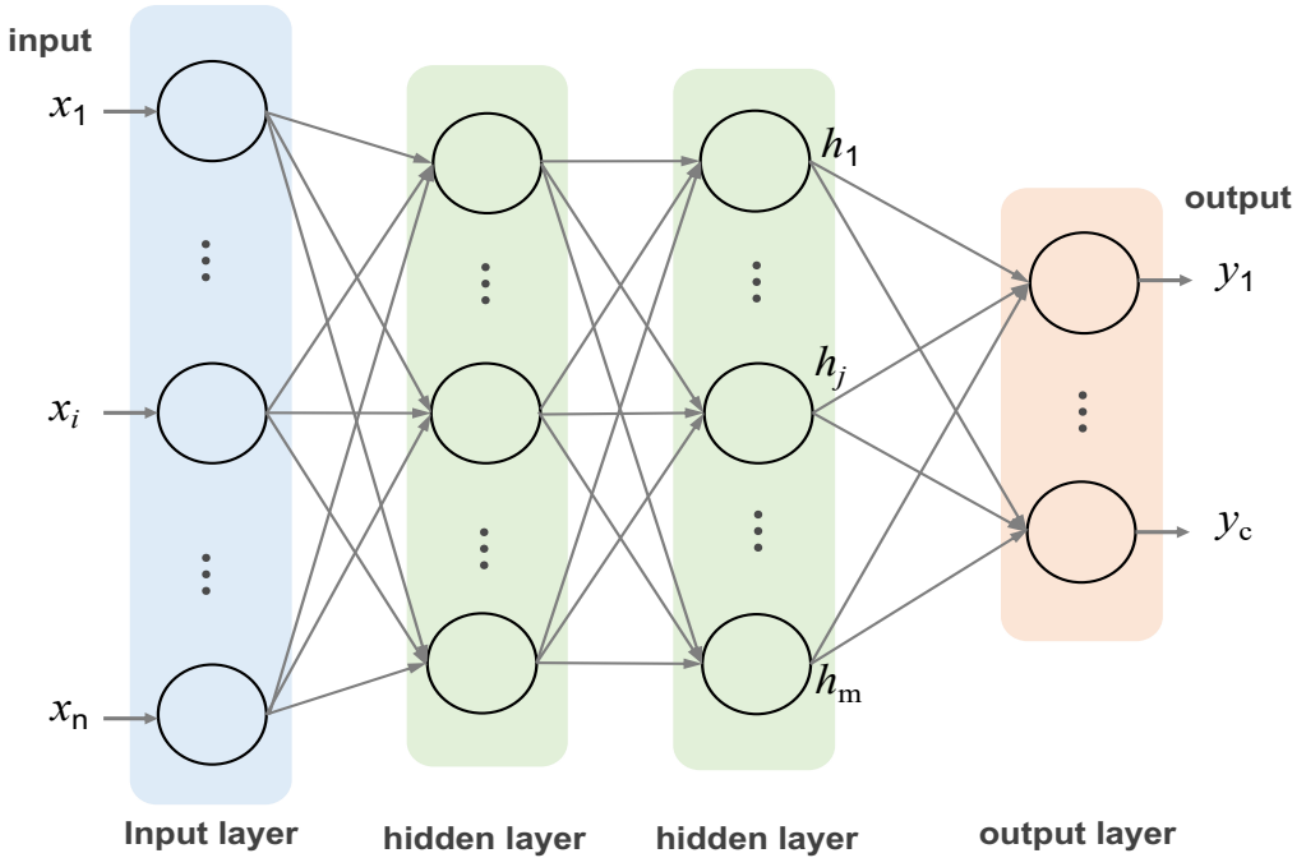
$h_2 = \text{step}(x_1 + x_2 - 1.5)$ [AND]

$y = \text{step}(h_1 - 2h_2 - 0.5)$

x_1	x_2	h_1	h_2	y
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

Multilayer Perceptrons

An MLP passes information from input to output through one or more hidden layers.



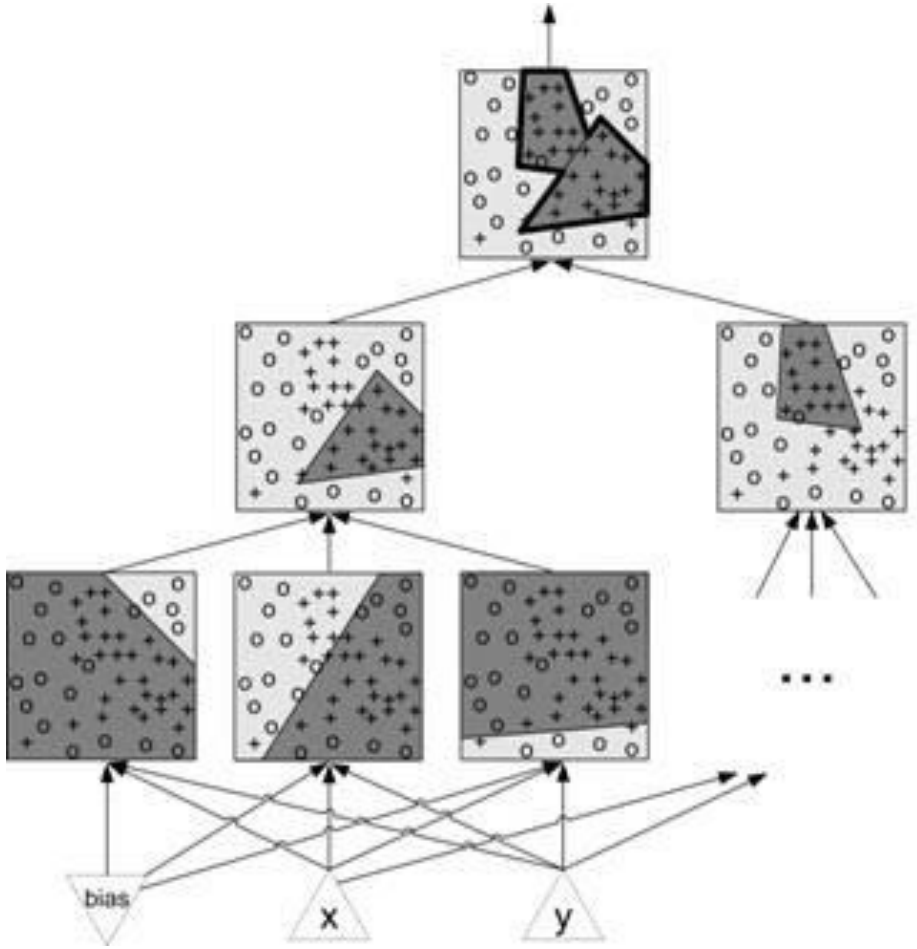
Nonlinear Hidden Units

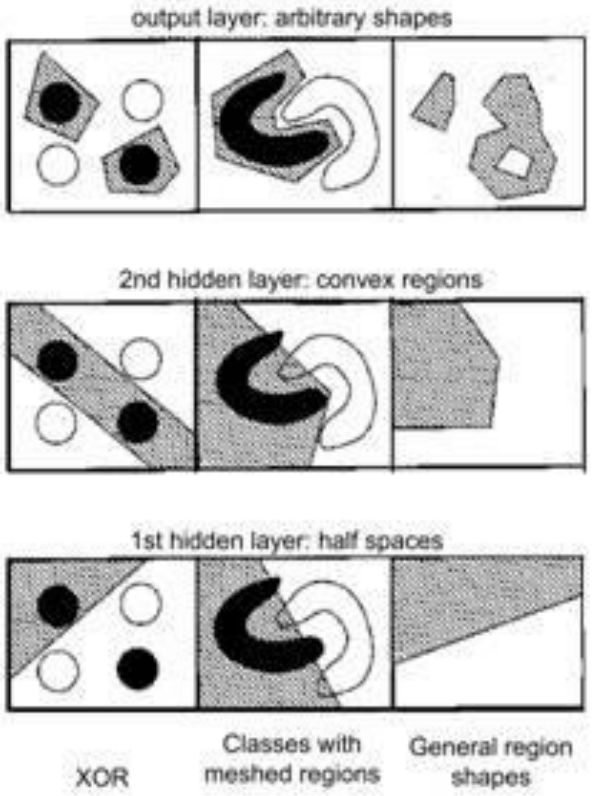
- Each unit forms a weighted sum, adds a bias, then applies an activation function.
- Stacking only linear activations still gives an affine function of the input.
- With suitable nonlinear activations and enough hidden units, an MLP can approximate continuous functions on compact domains.

Representational capacity does not guarantee successful training.

Reference: Hornik, Stinchcombe & White, Neural Networks 2(5), 359–366, 1989.

Combining Decision Regions

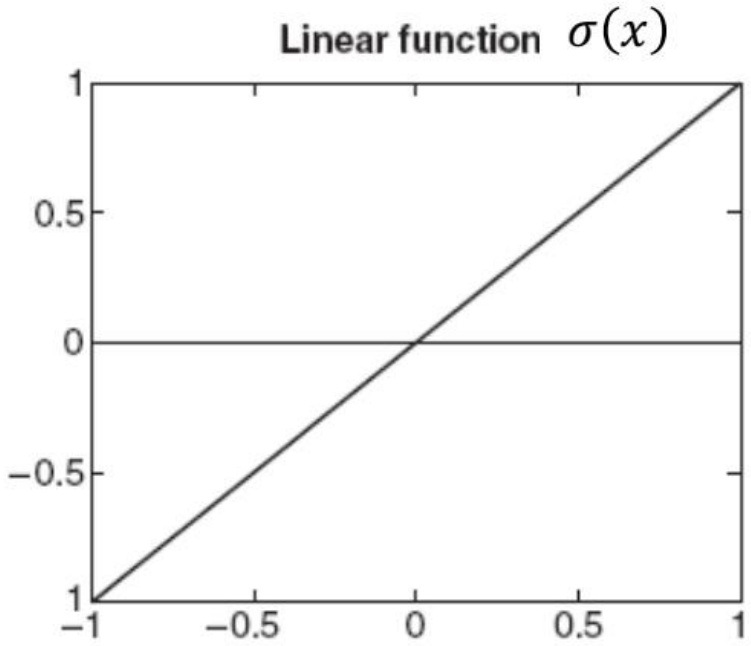


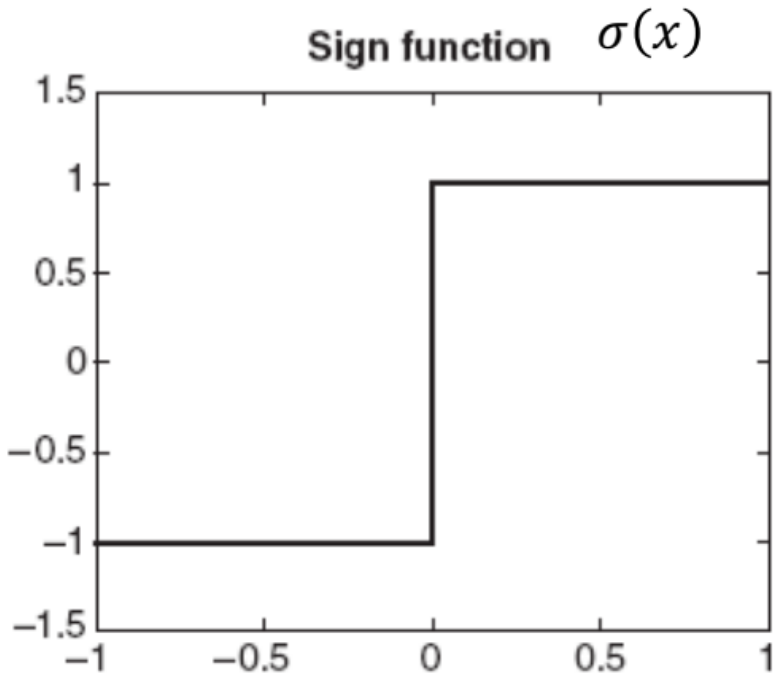


Hidden units combine simple boundaries into more complex regions.

Illustrations retained from the source slides. Original attribution: neural-forecasting.com

Activation Functions: Linear and Sign





$f(z) = z$

Useful for a real-valued output.

$f(z) = \text{sign}(z)$

Discrete output. Zero derivative away from the jump.

Activation Functions: Sigmoid and Tanh

$\sigma(z) = 1 / (1 + e^{-z})$

$f(z) = \tanh(z)$

z	Sigmoid σ(z)	Tanh(z)
−2	0.1192	−0.9640
−1	0.2689	−0.7616
0	0.5000	0.0000
1	0.7311	0.7616
2	0.8808	0.9640

Range: (0, 1)

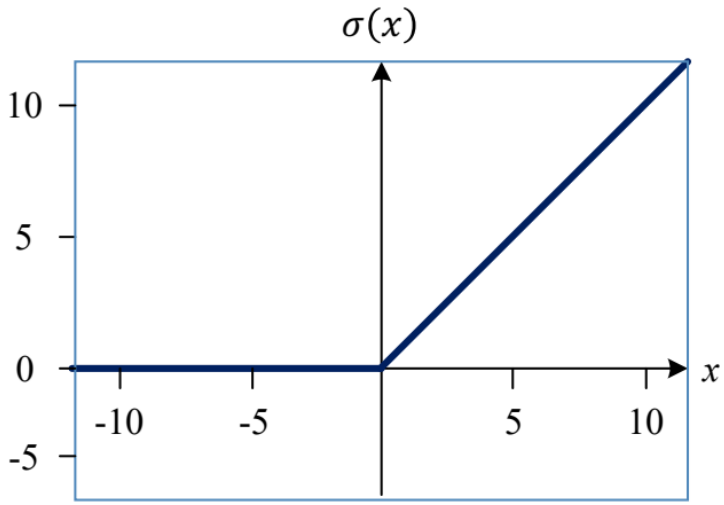
$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$

Range: (−1, 1)

$f'(z) = 1 - \tanh^2(z)$

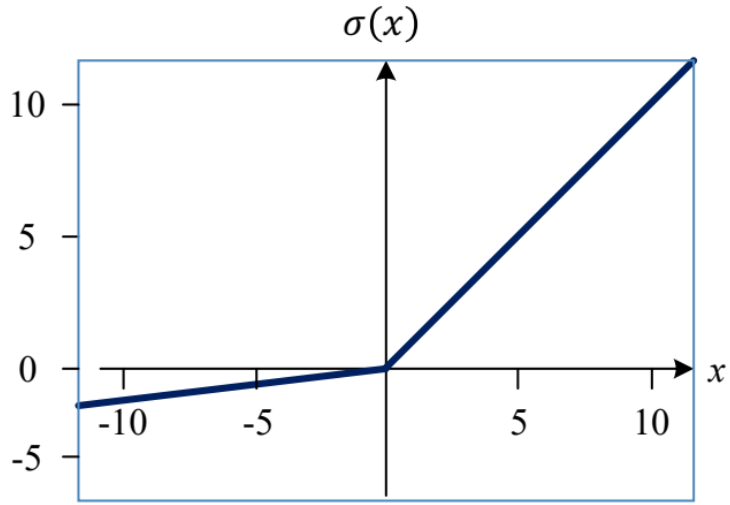
Activation Functions: ReLU

$f(z) = \max(0, z)$



ReLU (Rectified Linear Unit)

$f(z) = \max(az, z)$



Leaky ReLU

ReLU: slope 0 or 1.

Leaky ReLU: small positive a.

At z = 0, software chooses a derivative convention.

Forward Inference

- Keep the learned weights and biases fixed.
- Compute the hidden activations, then the output.

$$net_j = \sum_i w_{ji}\, o_i + b_j$$

$$o_j = \sigma(net_j)$$

For the first hidden layer, o_i is the input x_i.

w_{ji} means the weight from unit i to unit j.

Forward Example 2: Sigmoid MLP

Input x = (1, 2). Two hidden units and one output.

$$h_1 = \sigma(x_1 - 0.5x_2)$$

$$h_2 = \sigma(0.5x_1 + 0.5x_2 - 0.5)$$

$$o = \sigma(h_1 - h_2)$$

Question: What output does this network produce?

Forward Example 2: Hidden Layer

$$\text{net}_1 = 1(1) - 0.5(2) = 0$$

$$h_1 = \sigma(0) = 0.5000$$

$$\text{net}_2 = 0.5(1) + 0.5(2) - 0.5 = 1$$

$$h_2 = \sigma(1) = 0.7311$$

Pass these activations to the output layer.

Forward Example 2: Output

$$\text{net}_o = 1(0.5000) - 1(0.7311) = -0.2311$$

$$o = \sigma(-0.2311) = 0.4425$$

With a class threshold of 0.5, predict class 0.

The activation function and the class threshold have different roles.

Inference uses no target value and makes no parameter update.

Forward Example 3: ReLU MLP

Input x = (2, −1). ReLU hidden units, linear output.

$$h_1 = \mathrm{ReLU}(x_1 + x_2 - 0.5) = 0.5$$

$$h_2 = \mathrm{ReLU}(-x_1 + 2x_2 + 1) = 0$$

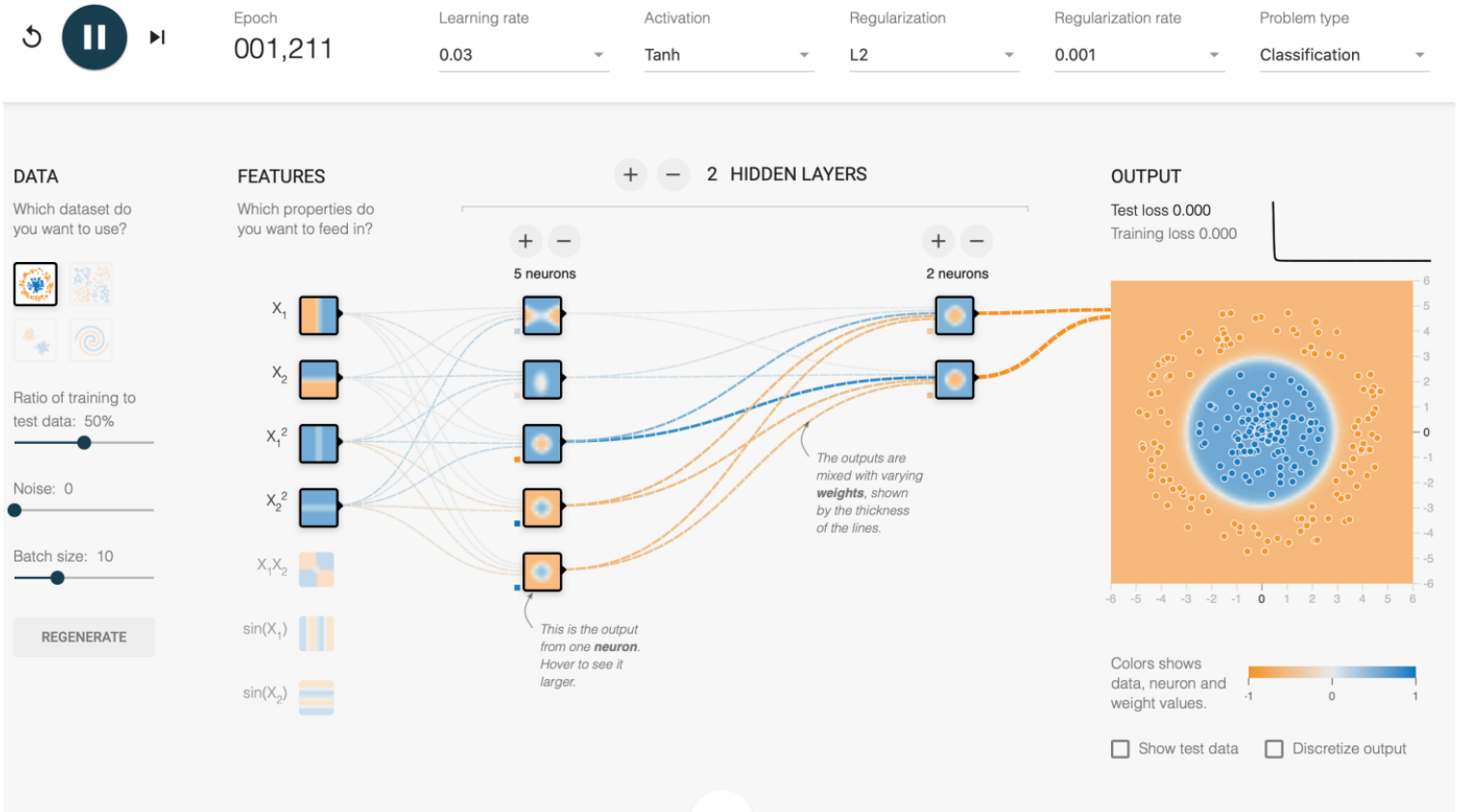
$$o = 2h_1 - h_2 + 0.1 = 1.1$$

The second hidden unit is inactive for this input.

A linear output can take values outside [0, 1].

Demo: TensorFlow Playground

Compare a network with no hidden layer to one with a nonlinear hidden layer on the circle dataset.



https://playground.tensorflow.org/

Learning from a Target

- Forward inference produces the network output o.
- A loss function compares o with the target t.
- Backpropagation computes gradients. An optimizer uses them to update the weights and biases.

$$E = \tfrac{1}{2} \sum_k (t_k - o_k)^2$$

We use squared error throughout the worked training examples.

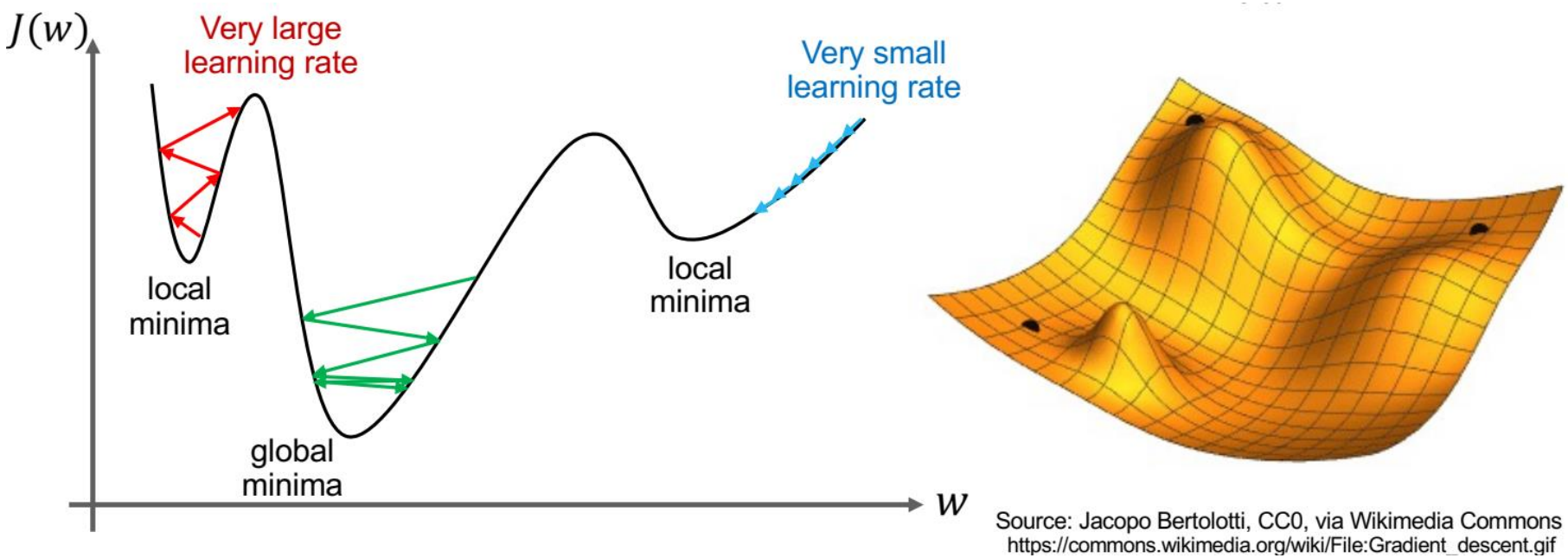
Gradient Descent

Move each parameter in the direction that reduces the loss locally.

$$w_{new} = w - \eta \, \partial E / \partial w$$

- The learning rate η controls the update size.
- A positive gradient makes the parameter decrease.
- A negative gradient makes the parameter increase.

The Learning Rate



Source: Jacopo Bertolotti, CC0, via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Gradient_descent.gif

- Too large: updates may overshoot or diverge.
- Too small: progress may be slow.
- Neural-network training has no general guarantee of reaching a global minimum.

Backpropagation: Setup and Forward Pass

- Initialize weights to small random values. Choose initial biases.
- For a training input, compute every net input and activation.
- Store these forward values for the backward pass.

$$net_j = \sum_i w_{ji}\, o_i + b_j$$

$$o_j = \sigma(net_j)$$

Backpropagation: Output Error Signal

For squared error and output unit k:

$$\delta_k = (t_k - o_k)\, \sigma'(net_k)$$

For a sigmoid output:

$$\delta_k = (t_k - o_k)\, o_k\, (1 - o_k)$$

Sign convention: $\delta_k = -\partial E / \partial net_k$. Updates will use $+\eta\delta$.

Backpropagation: Hidden Error Signal

A hidden unit receives error signals from the next layer.

$$\delta_j = \sigma'(\text{net}_j) \sum_k w_{kj}\, \delta_k$$

- Weight each downstream error signal by its connection weight.
- Add the contributions, then multiply by the local activation derivative.

Use the weights from the forward pass, before any update.

Backpropagation: Parameter Updates

$$\Delta w_{ji} = \eta \, \delta_j \, o_i$$

$$w_{ji,new} = w_{ji} + \Delta w_{ji}$$

$$\Delta b_j = \eta \, \delta_j$$

$$b_{j,new} = b_j + \Delta b_j$$

Compute all error signals first. Then update all parameters.

When to Update Parameters

Case updating (stochastic gradient descent)

One training example per update.

Mini-batch updating

Average the gradients over a small batch, then update.

Epoch updating (full-batch gradient descent)

Use the whole training set for one update.

An epoch is one pass through the training set.

When to Stop Training

- The loss reaches a chosen threshold.
- Parameter updates become sufficiently small.
- A maximum number of epochs or updates is reached.
- A validation criterion can help identify overfitting.

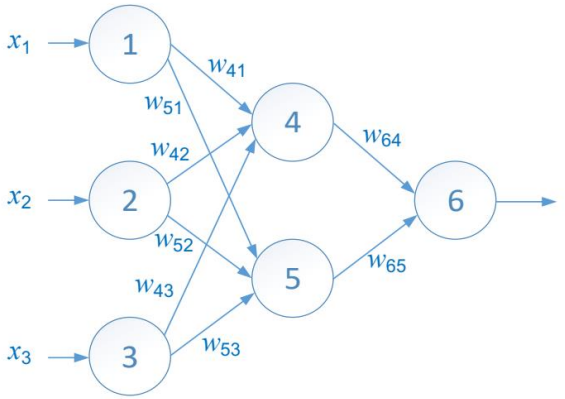
Small training error alone does not establish generalization.

Backpropagation Example 1: Setup

x = (1, 0, 1), target t = 1, learning rate η = 0.9

Sigmoid activation at hidden units 4, 5 and output unit 6.

Unit	Incoming weights	Bias
4	0.2, 0.4, −0.5	−0.4
5	−0.3, 0.1, 0.2	0.2
6	−0.3, −0.2	0.1



Weights follow input order
(1, 2, 3), then hidden order (4, 5).

Example 1: Hidden Activations

$$\text{net}_4 = 0.2(1) + 0.4(0) - 0.5(1) - 0.4$$

$$\text{net}_4 = -0.7 \qquad o_4 = \sigma(-0.7) = 0.3318$$

$$\text{net}_5 = -0.3(1) + 0.1(0) + 0.2(1) + 0.2$$

$$\text{net}_5 = 0.1 \qquad o_5 = \sigma(0.1) = 0.5250$$

The input $x_2 = 0$ contributes nothing to either weighted sum.

Example 1: Output and Loss

$$net_6 = -0.3(0.3318) - 0.2(0.5250) + 0.1$$

$$net_6 = -0.1045$$

$$o_6 = \sigma(net_6) = 0.4739$$

$$E = \tfrac{1}{2}(1 - 0.4739)^2 = 0.1384$$

Target = 1. The current output is below the target.

Example 1: Backward Error Signals

$$\delta_6 = (1 - 0.4739)(0.4739)(1 - 0.4739)$$

$$\delta_6 = 0.13117$$

$$\delta_k = (t_k - o_k)\, o_k\, (1 - o_k)$$

$$\delta_4 = (0.3318)(1 - 0.3318)(-0.3)(0.13117)$$

$$\delta_4 = -0.008725$$

$$\delta_j = \sigma'(net_j) \sum_k w_{kj}\, \delta_k$$

$$\delta_5 = (0.5250)(1 - 0.5250)(-0.2)(0.13117)$$

$$\delta_5 = -0.006542$$

Example 1: Output-Layer Updates

Learning rate η = 0.9

$$w_{64,new} = -0.3 + 0.9(0.13117)(0.3318)$$

$$w_{64,new} = -0.26083$$

$$\Delta w_{ji} = \eta\, \delta_j\, o_i$$

$$w_{ji,new} = w_{ji} + \Delta w_{ji}$$

$$w_{65,new} = -0.2 + 0.9(0.13117)(0.5250)$$

$$w_{65,new} = -0.13803$$

$$b_{6,new} = 0.1 + 0.9(0.13117) = 0.21805$$

$$\Delta b_j = \eta\, \delta_j$$

$$b_{j,new} = b_j + \Delta b_j$$

Example 1: Hidden-Layer Updates

$\Delta w_{ji} = 0.9\delta_j x_i$ and $\Delta b_j = 0.9\delta_j$

Parameter	Unit 4	Unit 5
Weight from x_1	0.19215	−0.30589
Weight from x_2	0.40000	0.10000
Weight from x_3	−0.50785	0.19411
Bias	−0.40785	0.19411

The two weights from x_2 do not change because $x_2 = 0$.

Example 1: A New Forward Pass

Use all updated weights and biases on the same input.

Quantity	Before update	After update
Hidden output o_4	0.3318	0.3266
Hidden output o_5	0.5250	0.5206
Output o_6	0.4739	0.5152
Squared-error loss	0.1384	0.1175

This update lowers the loss on this training example.

Backpropagation Example 2: A Small MLP

One input, one hidden unit and one output. x = 1, t = 1.

$$h = \sigma(wx + b_h) \qquad o = \sigma(vh + b_o)$$

Initial values: $w = 0$, $b_h = 0$, $v = 1$, $b_o = 0$, $\eta = 0.5$

$$h = \sigma(0) = 0.5 \qquad o = \sigma(0.5) = 0.62246$$

$$E = \tfrac{1}{2}(1 - o)^2 = 0.07127$$

Example 2: Output Error and Updates

$$\delta_o = (1 - o)o(1 - o) = 0.08872$$

$$\Delta v = \eta\, \delta_o\, h = 0.5(0.08872)(0.5)$$

$$v_{new} = 1 + 0.02218 = 1.02218$$

$$b_{o,new} = 0 + 0.5(0.08872) = 0.04436$$

The output bias receives the same error signal, without h.

Example 2: Hidden Error and Updates

$$\delta_h = h(1 - h)\, v\, \delta_o$$

$$\delta_h = (0.5)(0.5)(1)(0.08872) = 0.02218$$

The backward pass uses the old value v = 1.

$$w_{new} = 0 + 0.5(0.02218)(1) = 0.01109$$

$$b_{h,new} = 0 + 0.5(0.02218) = 0.01109$$

Example 2: Checking the Update

$$h_{new} = \sigma(0.01109 + 0.01109) = 0.50554$$

$$o_{new} = \sigma(1.02218 \times 0.50554 + 0.04436)$$

$$o_{new} = 0.63671$$

	Before	After
Output	0.62246	0.63671
Loss	0.07127	0.06599

Backpropagation Example 3: ReLU

Reuse Forward Example 3: $x = (2, -1)$, $h_1 = 0.5$, $h_2 = 0$.

Linear output $o = 2h_1 - h_2 + 0.1 = 1.1$. Let $t = 1$, $\eta = 0.1$.

$$\delta_o = (t - o) \times 1 = -0.1$$

$$\delta_1 = 1 \times 2 \times (-0.1) = -0.2$$

$$\delta_2 = 0 \times (-1) \times (-0.1) = 0$$

$$h_1 = \mathrm{ReLU}(x_1 + x_2 - 0.5) = 0.5$$

$$h_2 = \mathrm{ReLU}(-x_1 + 2x_2 + 1) = 0$$

$$o = 2h_1 - h_2 + 0.1 = 1.1$$

The inactive ReLU has zero local derivative for this input.

Example 3: Active and Inactive Paths

Parameter	Before	After
h_1 weights from (x_1, x_2)	(1, 1)	(0.96, 1.02)
h_1 bias	−0.5	−0.52
Output weights from (h_1, h_2)	(2, −1)	(1.995, −1)
Output bias	0.1	0.09

All parameters entering inactive h_2 stay unchanged.

$$h_{1,\text{new}} = 0.38 \qquad o_{\text{new}} = 0.8481$$

Loss rises: 0.00500 to 0.01154. This learning rate is too large for this step.

Output-Weight Derivation

Optional: applying the chain rule

$$\partial E/\partial w_{kj} = (\partial E/\partial o_k)(\partial o_k/\partial net_k)(\partial net_k/\partial w_{kj})$$

$$\partial E/\partial w_{kj} = (o_k - t_k)\ \sigma'(net_k)\ o_j$$

$$\partial E/\partial w_{kj} = -\delta_k\ o_j$$

$$\Delta w_{kj} = -\eta\ \partial E/\partial w_{kj} = \eta\ \delta_k\ o_j$$

Hidden-Weight Derivation

Optional: add every path from hidden unit j to the outputs.

$$\partial E/\partial o_j = \sum_k (\partial E/\partial net_k)(\partial net_k/\partial o_j)$$

$$\partial E/\partial o_j = -\sum_k \delta_k \, w_{kj}$$

$$\partial E/\partial w_{ji} = (\partial E/\partial o_j) \, \sigma'(net_j) \, o_i$$

$$\partial E/\partial w_{ji} = -\delta_j \, o_i$$

The Sigmoid Derivative

Optional: why the factor o(1 − o) appears

$$\sigma(z) = 1 / (1 + e^{-z})$$

$$\sigma'(z) = e^{-z} / (1 + e^{-z})^2$$

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

When a sigmoid saturates near 0 or 1, its derivative is small.

Backpropagation: One Training Step

1. Forward pass

$$\text{net}_j = \sum_i w_{ji}\, o_i + b_j \qquad o_j = \sigma(\text{net}_j)$$

2. Backward pass

$$\delta_k = (t_k - o_k)\, \sigma'(\text{net}_k)$$

$$\delta_j = \sigma'(\text{net}_j) \sum_k w_{kj}\, \delta_k$$

3. Update, after all error signals are available

$$w_{ji} \leftarrow w_{ji} + \eta\, \delta_j\, o_i \qquad b_j \leftarrow b_j + \eta\, \delta_j$$

Practice: Forward and Backward

$x = 2$, $t = 0$. Let $h = \sigma(wx + b_h)$ and $o = vh + b_o$.

Initial parameters: $w = 0$, $b_h = 0$, $v = 2$, $b_o = 0$.

Use $\eta = 0.1$ and $E = \frac{1}{2}(t - o)^2$.

- Compute h, o and the loss.
- Find the output and hidden error signals.
- Update all four parameters.
- Predict whether the next loss will be smaller.

Practice: Worked Answer

$h = 0.5 \quad o = 1 \quad E = 0.5$

$\delta_o = -1 \quad \delta_h = (0.5)(0.5)(2)(-1) = -0.5$

Parameter	Update	New value
w	$0.1(-0.5)(2)$	-0.10
b_h	$0.1(-0.5)$	-0.05
v	$0.1(-1)(0.5)$	1.95
b_o	$0.1(-1)$	-0.10

$h_{new} = 0.4378 \quad o_{new} = 0.7538 \quad E_{new} = 0.2841$